Overview

Due Date: 06/19/2026

For CPG – Cloud Fundamentals Capstone, you will need to organize yourself into groups of at least 4 individuals. You will each need to contribute to each lab and individual's contributions will be scored both in terms of the team accomplishment and their own portion. This multi-part assignment is designed to strengthen your practical skills in:

- Collaborative software development using **Git CLI and GitHub**
- Writing and deploying **Linux/Bash startup scripts** in a cloud environment
- Server-Side Linux CLI navigation and interaction
- **Troubleshooting and debugging** real codebases

You will complete **three independent labs**, each targeting a different core competency used in professional software and cloud engineering workflows. *Each lab is submitted as a group repo, with each member's contributions evaluated separately.*

Submission Summary

You will submit:

1. **Lab 1**: GitHub repo link *(repo must contain the following):*
    a. **A nested directory for each group** member
    b. **Each directory should have its own README.md**
2. **Lab 2**: GitHub repo link *(repo must contain the following):*
    a. Startup script + screenshots
3. **Lab 3**: GitHub repo link *(repo follow the below format):*
    a. Forked original lab 3 rep
    b. Each group member will created a nested subdirectory and place their contributions or code changes within this directory.
    c. Robust documentation in markdown about what errors were encountered and how they were resolved. (each nested directory should have documentation created by that individual student)

Professional Skills

Reinforced By completing this

assignment, you will practice: ● Git collaboration

workflows

- Branching and merge conflict resolution
- Bash scripting and Linux automation
- Cloud infrastructure initialization
- Debugging unfamiliar codebases
- Technical documentation

Lab 1 — Branching, Git, & Python

Collaboration Objective

The goal of this lab is to practice **team-based Git workflows** while building a **simple, functional Python console application**. Emphasis is placed on **branching discipline, collaboration, and merge integration**, not application complexity.

Example repository: https://github.com/rossislike/2025dec22

Scenario:

You are working in a GitHub repository for a small Python project. Your goal is to clone the repository, create a feature branch, make code changes, update documentation, and push your work. Along the way, you will intentionally create common Git mistakes so you can recognize and understand how to mitigate them in real projects.

Technical Requirements

Repository Setup

- Create a **brand-new GitHub repository** from scratch (no templates).
- Repository must include:
    - .README.md
    - .gitignore (appropriate for Python)
    - At least 2 Python files (.py) per student

Git Workflow Requirements

Round 1: Happy Path

In this round you will demonstrate a base understanding of GitOps by successfully merging multiple branches back into main without any issues. In tech, the Happy Path

scenario is one in which no errors or issues present themselves.

Each team member must:

1. Clone the repository locally
2. Create **their own Git branch**
    - Naming convention (recommended):
      feature/<firstname>-function
3. Write **at least two Python files with the following code concepts implemented**
    - **Static and dynamic variables**
    - **User input**
    - **File creation**
    - **Conditional Logic**
4. Commit their work to their branch
5. Push their branch to GitHub
6. Open a **Pull Request** (PR)
7. Participate in resolving merge conflicts (if any)

No one should push directly to main except through Pull Requests.

Round 2: Exception Path

In this round you will demonstrate an intermediate understanding of GitOps by manually creating scenarios that require mitigation and then taking the steps to resolve them before merging back into the main branch. In tech, the Exception Path scenario are those in which errors or exceptions and arise. In order for the for the scenario to complete successfully, additional steps, troubleshooting, or error-handling must be utilized.

I. Stage Unwanted Files

Intentionally create extra files that should not be committed, such as:

- notes.txt
- debug.log
- test_output.tmp

Then stage all files in the directory. Run a git status to verify that all files in the directory, including extraneous files have been staged. You must remove these staged files before pushing your second batch of Python files to the main branch.

Deliverable:
You must keep these files in the local directory tied to your GitHub repo without committing or pushing them. Capture screenshots of your console and add them to a nested subdirectory labeled “1.2.1_Visuals”

Hint: Unwanted staged files can be removed from the staging area without deleting them locally. This is usually handled with Git restore or reset commands. The key idea is that the staging area is separate from the working directory.

II. Missing Commit message

Run a git commit command without adding a commit message This will create a downstream error that you will then need to resolve.

Deliverable:
You must resolve the empty commit message before pushing to your branch. Capture screenshots of your console commands and add them to a nested subdirectory labeled “1.2.2_Visuals”

Hint: Use a CLI-based text editor tool to manually add commit message

III. Create a Merge Conflict

To create a merge conflict there must be incompatible or overlapping code differences to a file that exists on the repo. To accomplish this, you may make changes to one of the files from round 1 or create a new shared file across the group repo.

Deliverable:
You must resolve the merge conflicts to overcome errors that will be thrown when you empty to push the updated code to your branch. Capture screenshots of your console commands and add them to a nested subdirectory labeled “1.2.3_Visuals”

Application Requirements

- Must be **console-based** (CLI)
- Must execute via:

```
python app.py
```

- Application purpose is **open-ended**

- Examples:
    - Calculator
    - Quiz game
    - Text-based menu
    - File utility

- Must demonstrate:
    - Function calls between files or modules
    - Conditional Logic
    - Variables (Both dynamic and static)
    - Basic user input/output
    - Clean code structure

Deliverables

- GitHub repository link

- Evidence of:
    - Multiple branches
        - 2 Separate branches and re-merges into main for each group member
    - Multiple contributors
    - Pull Requests

- Final merged working application on main

Evaluation Criteria

Area	**Weight**
Proper Git branching	30%
Individual contribution	25%
Successful merge to main	25%
Application runs without errors	20%

Lab 2 — EC2 Startup Script (User Data

Automation) **Objective**

This lab introduces **Linux automation and cloud initialization** by writing a startup script that configures an EC2 instance to host a **front-facing static website**.

You will use **Bash scripting** to automate server setup without manual intervention.

Technical Requirements

Environment

- AWS EC2 (Amazon Linux or Ubuntu)
- Script must be compatible with **EC2 User Data**
- No SSH configuration required for grading

Startup Script Requirements

Your script must:

- Be written in **Bash**
- Execute automatically at instance launch
- Perform the following tasks:

    1. Install a web server (Apache or Nginx)
    2. Create an HTML page
        - this must use a .html file and a .css file
    3. Serve content publicly via HTTP
    4. Create backup directories of all generated files
    5. Create a log – archive directory and generate 4 dummy files into this directory
    6. Create additional directories with placeholder files for the following:
        - content
        - utils
        - credentials

7. Once your EC2 instance is up and running, you will need to ssh into this server via the AWS Console and use Linux to validate your directory structures were created as intended

Web Page Content Requirements

The generated web page must include:

- **Your name**
- A **background color or image**
- At least **one embedded image**
- **Three distinct content sections**, such as:
    - About Me
    - Project Description
    - Contact / Footer

HTML and CSS must be separate files.

Deliverables

- Bash startup script file (startup.sh)
- Screenshot of:
    - Running EC2 instance
    - Rendered web page in browser
    - CLI activity on the EC2 Server
- Brief explanation of script logic

Evaluation Criteria

Area	Weight

Script runs without errors 30% Web

page renders correctly 20%

Required page elements present 20%

Script readability & comments 15%

Server structure 15%

Lab 3

Lab 3 — Troubleshooting & Debugging

Exercise Objective

This lab simulates **real-world debugging scenarios** where engineers must understand, fix, and execute unfamiliar codebases.

You will be assessed on your **problem-solving process**, not just the final

result. **Technical Requirements**

1. Fork the provided GitHub repository
2. Clone your fork locally
3. Attempt to run the application
4. Identify:
    - ○ Runtime errors
    - ○ Syntax errors
    - ○ Dependency issues
    - ○ Environment mismatches

Expected Debugging Actions

- ● Read error messages carefully

-
- Modify code incrementally
- Install missing dependencies if needed
- Fix:
    - Incorrect imports
    - Logic errors
    - Version mismatches

You **may not rewrite the project entirely** — the goal is targeted fixes.

Documentation Requirements

You must document:

- Initial error(s) encountered
- Steps taken to diagnose
- Code changes made
- Final successful execution

This may be included in:

- README.md
- Separate DEBUGGING.md file

Deliverables

- Link to your forked repository
- Updated, runnable code
- Written troubleshooting explanation

Evaluation Criteria

Area	Weight

Correct execution 40%

Debugging documentation 30%

Quality of fixes 20%
Commit clarity 10%